

PROJECT REPORT

EXPERIMENT – 6

Project title:

SMART ELEVATOR AUTOMATION AND HUMAN
DETECTION SYSTEM USING PIR SENSOR

Submitted by:

Arundepak S – 24MER004

Deepak P – 24MER006

Dharanidharan R S – 24MER007

Dinesh K – 24MER009

Vishnivarthan P – 24MER063

SMART ELEVATOR AUTOMATION AND HUMAN DETECTION SYSTEM USING PIR SENSOR

1. Project Overview

This project implements an IoT-based smart elevator automation system that uses a PIR (Passive Infrared) motion sensor to detect human presence and automatically control the elevator door using a servo motor. Built as a safety and automation module for lifts, the system removes the need for manual button pressing to open or close the door, reduces waiting time, and improves accessibility. LED indicators show door status, a buzzer gives audible alerts, and push buttons allow manual override. The system also sends live status data to the Adafruit IO cloud platform using the MQTT protocol, allowing remote monitoring through a dashboard.

The working prototype was built, debugged, and verified on an ESP8266 NodeMCU board interfaced with a PIR motion sensor, an SG90 servo motor, two LED indicators, a buzzer, and two push buttons, with cloud connectivity integrated via Adafruit IO for remote data logging and status visualization.

2. Problem Statement

Conventional elevator doors depend on manual button presses or simple timer-based auto-close mechanisms. This leads to:

- Delayed door response when hands are full or during emergencies.
- Doors closing on people because there is no real-time human detection.
- No remote visibility of door status or elevator activity for building managers.
- Lack of automation, making the system less accessible for elderly or differently-abled users.
- No safety alert (audio/visual) when the door is opening or closing.

There is a need for a low-cost, sensor-based automation system that detects human presence, safely operates the door, and reports live status to a cloud dashboard for monitoring.

3. Proposed Solution

A standalone IoT-based elevator automation node built around the ESP8266 NodeMCU is proposed and implemented with the following capabilities:

- Continuous human presence detection using a **PIR motion sensor**.
- Automatic door **open/close control** using an **SG90 servo motor** when motion is detected.
- **LED indicators** to visually show door status (open/closed).
- **Buzzer alert** during door movement for safety warning.
- **Push buttons** for manual door open/close override.
- Cloud data logging to **Adafruit IO** over Wi-Fi via MQTT, publishing door status, motion value, and buzzer/alert status to a live dashboard.

4. System Architecture

The system follows a layered IoT architecture:

- **Sensing Layer:** PIR sensor detects infrared body heat/motion near the elevator door.
- **Edge / Control Layer:** ESP8266 NodeMCU reads the PIR signal, runs the decision logic, and drives the servo motor, LEDs, and buzzer.
- **Connectivity Layer:** ESP8266 built-in Wi-Fi connects to a local access point and communicates over TCP/IP.
- **Cloud Layer:** Sensor and status data is published to an Adafruit IO channel using the MQTT protocol.
- **Visualization Layer:** Adafruit IO dashboard displays door status (ON/OFF toggle), motion value (gauge), and buzzer/event log in real time.

Data flow: PIR Sensor → NodeMCU (Logic) → Servo/LED/Buzzer (Local Action) + Adafruit IO (Cloud Update)

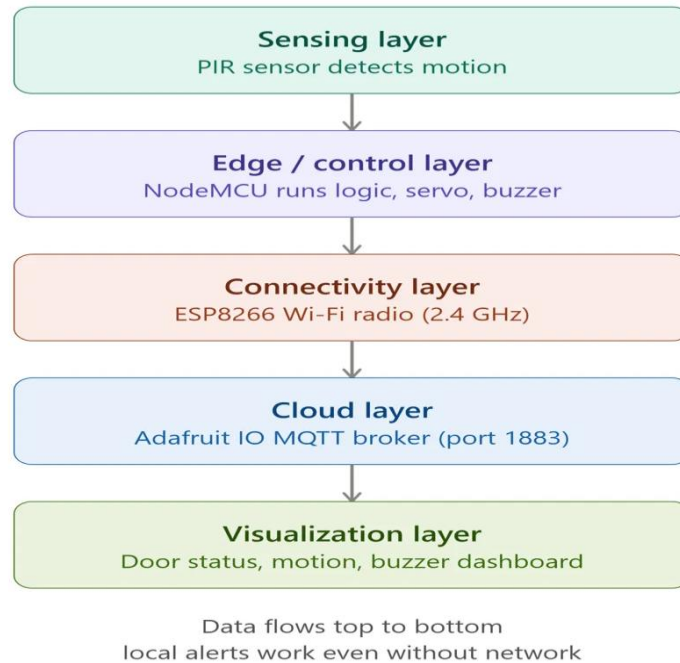


Fig. 1 Layered IoT system architecture

5. Circuit Table

Component	Pin	NodeMCU Pin	GPIO
PIR Sensor	VCC	3V3	–
PIR Sensor	OUT	D5	GPIO14
PIR Sensor	GND	G	–
Servo Motor (SG90)	Signal	D6	GPIO12
Servo Motor (SG90)	VCC	5V/VIN	–
Servo Motor (SG90)	GND	G	–

Component	Pin	NodeMCU Pin	GPIO
LED 1 (Door Open)	Anode	D1	GPIO5
LED 2 (Door Closed)	Anode	D2	GPIO4
Buzzer	I/O (Signal)	D7	GPIO13
Push Button 1 (Open)	Signal	D3	GPIO0
Push Button 2 (Close)	Signal	D4	GPIO2
All GND	GND	G	–

6. Circuit Design

The circuit is designed around the ESP8266 NodeMCU development board powered via a standard USB connection (5V DC):

- The **PIR sensor** output is connected to a digital GPIO pin; power is drawn from the 3.3V rail.
- The **servo motor** signal wire connects to a PWM-capable GPIO pin; power is drawn from the VIN/5V rail to avoid brownout of the NodeMCU.
- **LED indicators** are connected through current-limiting resistors to show door open/closed status.
- The **buzzer** is directly driven by a GPIO pin via digital logic switching (HIGH/LOW).
- **Push buttons** are connected with internal pull-up resistors for manual override.
- All ground pins are commoned across the prototyping breadboard rail.

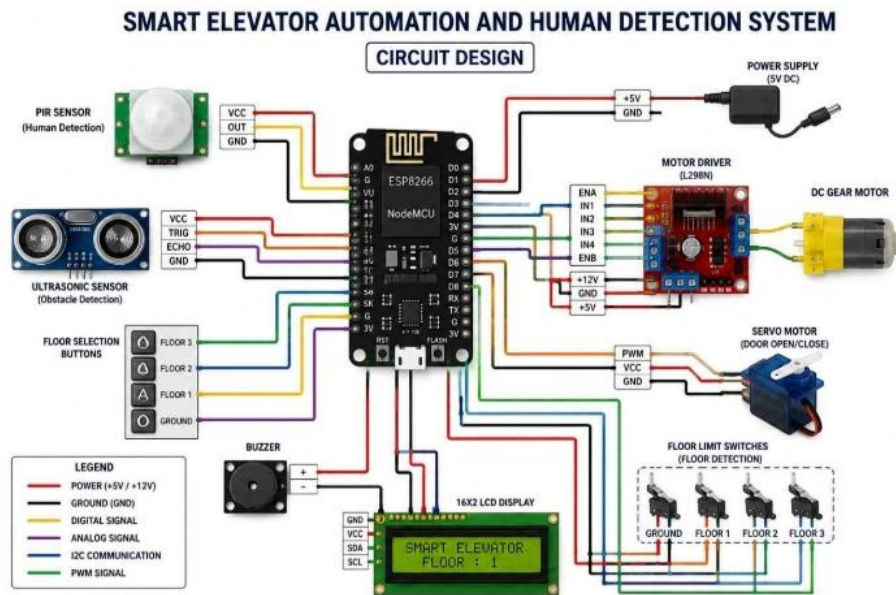


Fig. 2 Circuit wiring diagram

7. Algorithm

- ☐ Initialize Serial communication (115200 baud), servo motor, PIR input pin, LED output pins, buzzer pin, and push button pins.
- ☐ Connect to the local Wi-Fi network and initialize the Adafruit IO MQTT client.
- ☐ Display "System Ready – Warming up PIR sensor..." and wait a few seconds for the PIR sensor to stabilize.
- ☐ Once ready, print "System Active – Monitoring for motion" and enter the main loop.
- ☐ In the main loop, continuously read the PIR sensor output.
- ☐ **If motion is detected:**
 - Print "Human detected – Door Opening".
 - Rotate the servo motor to the open position.
 - Turn ON the "Door Open" LED and turn OFF the "Door Closed" LED.

- Sound the buzzer briefly as a safety alert.
 - Hold the door open for a fixed delay to allow entry/exit.
- **After the hold time (no new motion):**
- Print "Door Closed".
 - Rotate the servo motor back to the closed position.
 - Turn ON the "Door Closed" LED and turn OFF the "Door Open" LED.
- Check push button states: Button 1 manually forces the door open; Button 2 manually forces the door closed, overriding the PIR logic.
- Publish door status, motion value, and buzzer status to Adafruit IO feeds at regular intervals using MQTT.
- Maintain the MQTT connection using the Adafruit IO client loop.

8. Coding

```
#include <ESP32Servo.h>

#include <WiFi.h>

#include "Adafruit_MQTT.h"

#include "Adafruit_MQTT_Client.h"


// ----- WiFi Credentials -----

#define WLAN_SSID    "test"

#define WLAN_PASS    "test1234"


// ----- Adafruit IO Credentials -----

#define AIO_SERVER    "io.adafruit.com"
```

```
#define AIO_SERVERPORT 1883
```

```
#define AIO_USERNAME  "your_username"
```

```
#define AIO_KEY      "your_aio_key"
```

```
WiFiClient client;
```

```
Adafruit_MQTT_Client  mqtt(&client,  AIO_SERVER,  AIO_SERVERPORT,  
AIO_USERNAME, AIO_KEY);
```

```
// Feeds
```

```
Adafruit_MQTT_Publish  doorStatusFeed  =  Adafruit_MQTT_Publish(&mqtt,  
AIO_USERNAME "/feeds/elevator-door-status");
```

```
Adafruit_MQTT_Publish  motionFeed      =  Adafruit_MQTT_Publish(&mqtt,  
AIO_USERNAME "/feeds/elevator-motion");
```

```
Adafruit_MQTT_Publish  buzzerFeed      =  Adafruit_MQTT_Publish(&mqtt,  
AIO_USERNAME "/feeds/elevator-buzzer");
```

```
#define PIR_PIN  14  // D5
```

```
#define SERVO_PIN 12  // D6
```

```
#define LED_OPEN  5  // D1
```

```
#define LED_CLOSE 4  // D2
```

```
#define BUZZER_PIN 13  // D7
```

```
#define BTN_OPEN  0  // D3
```

```
#define BTN_CLOSE 2  // D4
```

```
Servo doorServo;
```

```
bool doorOpen = false;
```



```
void connectWiFi() {  
  
    Serial.print("Connecting to WiFi.....");  
  
    WiFi.begin(WLAN_SSID, WLAN_PASS);  
  
    while (WiFi.status() != WL_CONNECTED) {  
  
        delay(500);  
  
        Serial.print(".");  
  
    }  
  
    Serial.println();  
  
    Serial.print("WiFi connected. IP: ");  
  
    Serial.println(WiFi.localIP());  
  
}  
  
void connectMQTT() {  
  
    Serial.print("Connecting to Adafruit IO...");  
  
    int8_t ret;  
  
    while ((ret = mqtt.connect()) != 0) {  
  
        Serial.println(mqtt.connectErrorString(ret));  
  
        mqtt.disconnect();  
  
        delay(3000);  
  
    }  
  
    Serial.println(" Adafruit IO Connected!");  
  
}  
  
void setup() {  
  
    Serial.begin(115200);
```

```
pinMode(PIR_PIN, INPUT);
```

```
pinMode(LED_OPEN, OUTPUT);
```

```
pinMode(LED_CLOSE, OUTPUT);
```

```
pinMode(BUZZER_PIN, OUTPUT);
```

```
pinMode(BTN_OPEN, INPUT_PULLUP);
```

```
pinMode(BTN_CLOSE, INPUT_PULLUP);
```

```
doorServo.attach(SERVO_PIN);
```

```
doorServo.write(0); // start closed
```

```
digitalWrite(LED_CLOSE, HIGH);
```

```
connectWiFi();
```

```
Serial.println("System Ready - Warming up PIR sensor...");
```

```
delay(20000); // let PIR stabilize
```

```
Serial.println("System Active - Monitoring for motion");
```

```
connectMQTT();
```

```
}
```

```
void openDoor() {
```

```
Serial.println("Human detected - Door Opening");
```

```
doorServo.write(90);
```

```
digitalWrite(LED_OPEN, HIGH);
```

```
digitalWrite(LED_CLOSE, LOW);
```

```
digitalWrite(BUZZER_PIN, HIGH);

delay(300);

digitalWrite(BUZZER_PIN, LOW);

doorOpen = true;
}


void closeDoor() {

  Serial.println("Door Closed");

  doorServo.write(0);

  digitalWrite(LED_OPEN, LOW);

  digitalWrite(LED_CLOSE, HIGH);

  doorOpen = false;
}

void loop() {

  if (!mqtt.connected()) connectMQTT();

  int motion = digitalRead(PIR_PIN);

  bool manualOpen = (digitalRead(BTN_OPEN) == LOW);

  bool manualClose = (digitalRead(BTN_CLOSE) == LOW);


  if (motion == HIGH || manualOpen) {

    openDoor();

    delay(4000); // door hold time

    if (!manualClose) closeDoor();
```

```
} else if (manualClose) {  
  
    closeDoor();  
  
}  
  
// Publish to Adafruit IO  
  
doorStatusFeed.publish(doorOpen ? 1 : 0);  
  
motionFeed.publish(motion);  
  
buzzerFeed.publish(motion == HIGH ? "ALERT" : "NORMAL");  
  
mqtt.processPackets(1000);  
  
}
```

9. System Working

On power-up, the ESP8266 NodeMCU initializes all GPIO pins, connects to the local Wi-Fi network, and warms up the PIR sensor for stable readings. It then runs a continuous monitoring loop:

- The PIR sensor continuously checks for human motion near the elevator entrance.
- If motion is detected (or the open button is pressed), the servo rotates to open the door, the "Door Open" LED turns ON, and the buzzer gives a short safety beep.
- The door stays open for a fixed hold time to allow safe entry/exit.
- If no further motion is detected, the servo rotates back, closing the door and switching the LED indicator to "Door Closed".
- The close button allows manual override to shut the door immediately.
- Door status, motion value, and buzzer status are continuously published to the **Adafruit IO** dashboard, updating the door status toggle, motion gauge, and event log in real time.

10. Bill of Materials

Component	Quantity	Purpose
ESP8266 NodeMCU	1	Main microcontroller with built-in Wi-Fi
PIR Motion Sensor	1	Detects human presence near the elevator door
Servo Motor (SG90)	1	Opens and closes the elevator door
LED Indicators	2	Shows door open/closed status
Buzzer	1	Audible safety alert during door movement
Push Buttons	2	Manual override for door open/close
Breadboard	1	Prototyping platform
Jumper Wires	1 set	Connections

11. Prototype Implementation

The prototype was assembled and debugged iteratively, with the following notable steps and issues resolved during implementation:

- Upload Port Error (PermissionError 13): Initial upload attempts on the Windows machine failed with a COM port permission error. Resolved by closing conflicting Serial Monitor sessions, changing the USB cable, and reselecting the correct board/port (ESP32 Dev Module on COM4).
- Servo Power Instability: Directly powering the servo from the NodeMCU 3.3V pin caused resets during rotation. Fixed by powering the servo from the 5V/VIN rail with a common ground.

- PIR False Triggers: Early testing showed false motion triggers right after power-up. Resolved by adding a warm-up delay ("Warming up PIR sensor...") before entering the monitoring state.
- MQTT Connection to Adafruit IO: Initial publish attempts failed due to incorrect feed names. Resolved by creating dedicated feeds (elevator-door-status, elevator-motion, elevator-buzzer) in the Adafruit IO dashboard and matching them exactly in code.
- Adafruit IO Throttle Warning: Frequent publishing triggered a "data rate limit reached" throttle warning. Resolved by increasing the publish interval between MQTT updates.

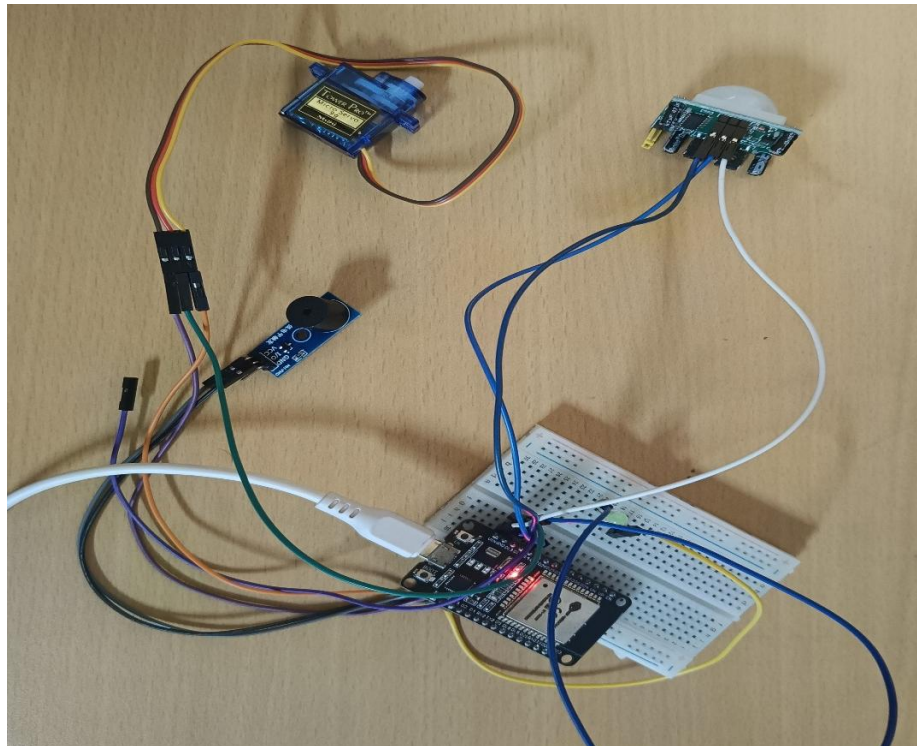


Fig. 3 Assembled prototype (idle state)

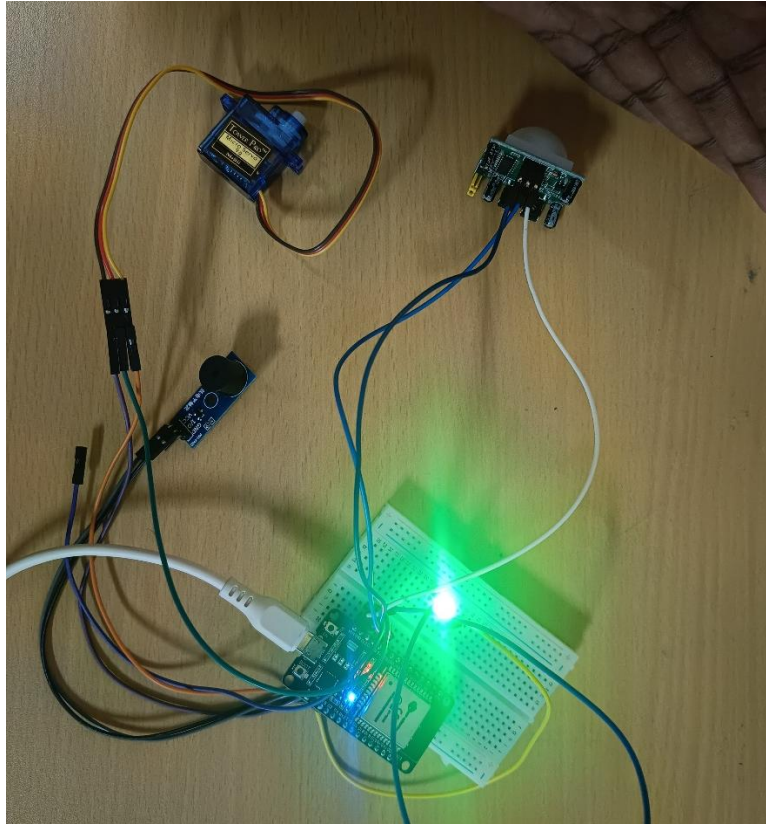


Fig. 4 Assembled prototype (motion test, LED green)

12. Results & Performance Analysis

Once wiring and configuration issues were resolved, the system performed as designed:

- **Sensor Responsiveness:** The PIR sensor reliably detected human presence, triggering door action within about 1–2 seconds of motion.
- **Local Alert Performance:** The servo motor, LEDs, and buzzer responded synchronously — door opened smoothly on detection and closed automatically after the hold time.
- **Local UI Verification:** LED indicators clearly showed door open/closed status without flicker; manual push buttons correctly overrode automatic operation.
- **Cloud Telemetry:** Status updates were published successfully to Adafruit IO over MQTT. The dashboard's **door-status toggle**, **motion gauge**, and **buzzer event log** updated correctly in real time, confirming reliable cloud connectivity.

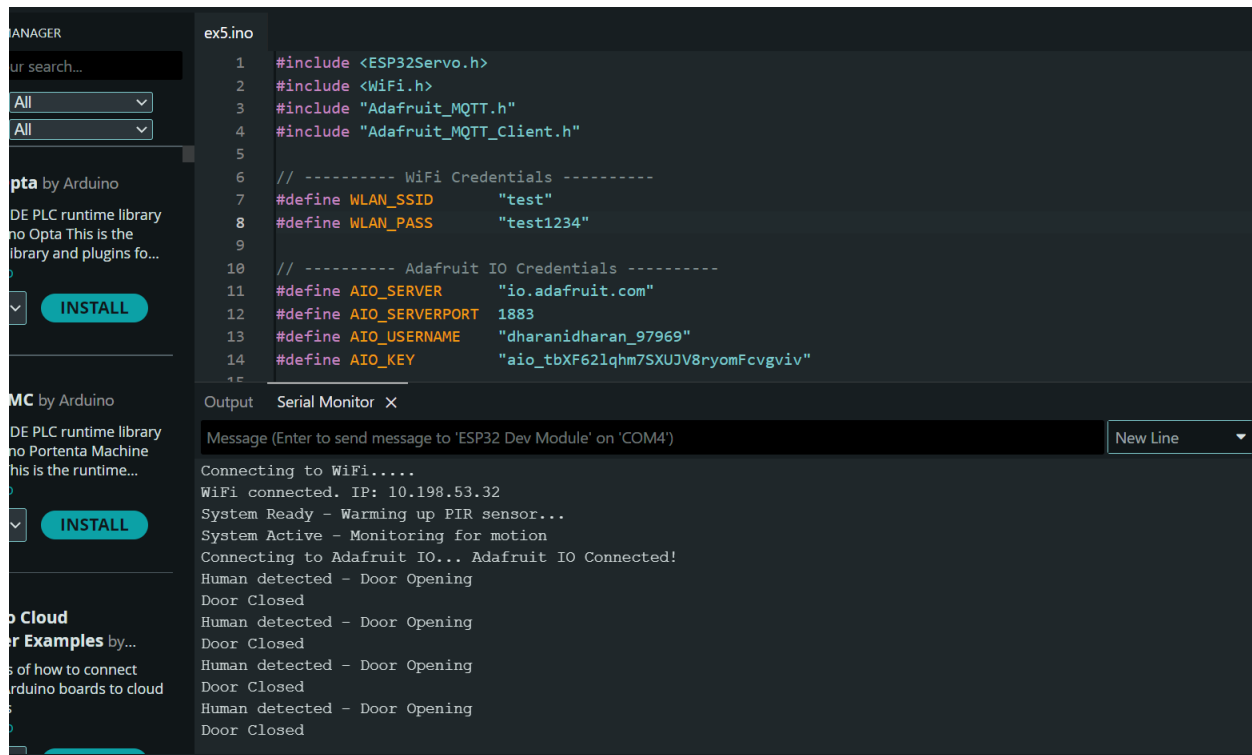


Fig. 5 Arduino IDE code + Serial Monitor output

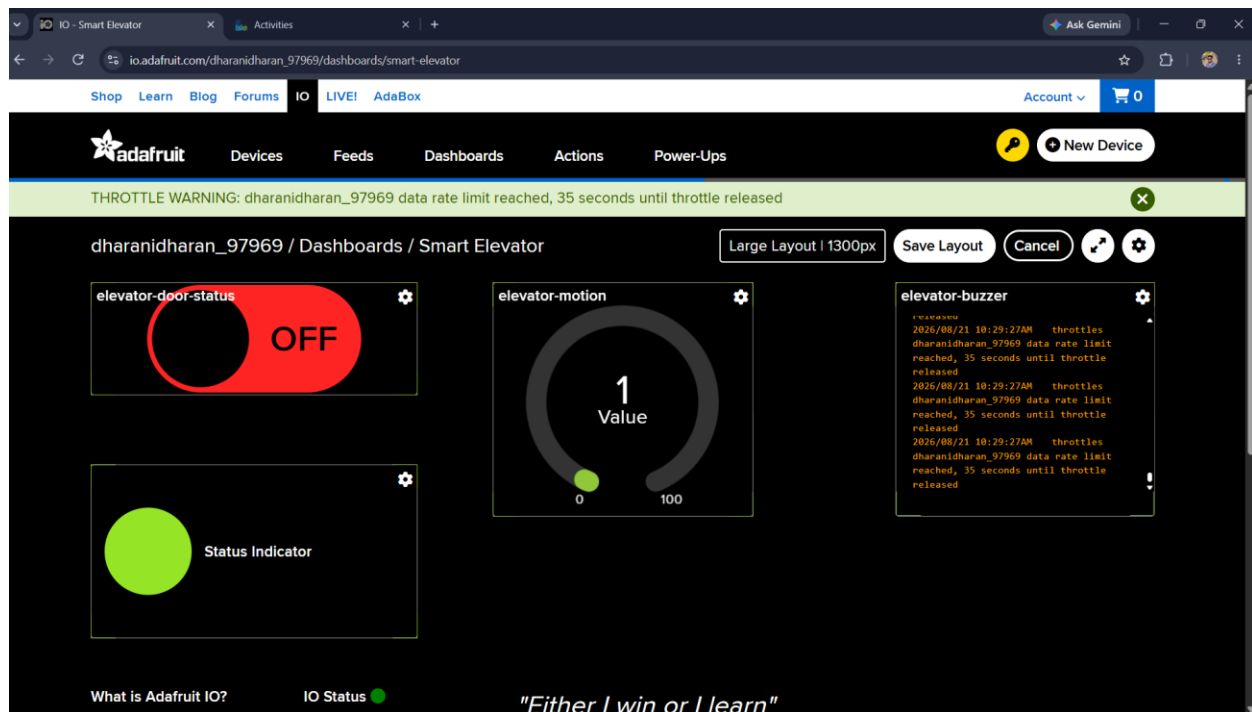


Fig. 6 Adafruit IO Dashboard